

Merge, Join & Concat

Day 18 — combining tables, and the rows that do not match

90 min

Concepts + live code

30 min

Practice notebook

4

Kinds of join



Boston University

Before we start — five questions from Day 17

1 What does the group key become?

→ The index of the result.

2 Sorted by value or by key?

→ By key. Sort the result yourself.

3 size() vs count()?

→ Rows vs non-missing values.

4 agg or transform for “above own group”?

→ transform — it keeps the shape.

5 What happens to a missing group key?

→ The row is dropped, silently.

Yesterday reshaped one table. Today combines two — and question 5's lesson returns in a new form.

How today's two hours are structured

1

The idea of a join

One key, two tables

15 min

2

The four joins

inner, left, right, outer

30 min

3

Merge in practice

Different names, suffixes, indicator

20 min

4

Duplicate keys & concat

Row explosion, and stacking tables

20 min

5

Mini build

Three tables into one report

5 min

6

Practice notebook

You code, I walk around

30 min

Section 4 contains the bug that silently multiplies your data. It is the reason to check row counts after every merge.

By the end of today you will be able to

1

Combine

Join two tables on a shared key

2

Choose

Pick inner, left, right or outer —
and justify it

3

Audit

Say what happened to every row
that did not match

4

Stack

Append tables with concat, and
know when not to

The question that matters in every merge

Not “which rows match?” — pandas handles that. The question is what should happen to the rows that DON'T. Dropping a customer with no orders, or keeping them with a blank total, are different business answers to the same data. The join type is that decision, written in code.

1

The Idea of a Join

One key, two tables · 15 minutes

- Why data lives in pieces
- The shared key

Real data arrives in pieces

students

	id	name	city
0	1	Ravi	Pune
1	2	Sara	Mumbai
2	3	Amit	Delhi
3	4	Neha	Pune

marks

	id	python
0	1	88
1	2	91
2	3	45
3	5	70

students.merge(marks, on="id")

	id	name	python
0	1	Ravi	88
1	2	Sara	91
2	3	Amit	45

id 4 has no marks. id 5 has no student. Both vanished.

The shared key is what makes it possible

Storing names once in a student table, rather than repeating them beside every mark, is how databases avoid duplication and contradictions. The price is that answering a real question needs two tables put back together — and that is a merge. You met the same idea on Day 9 when a CSV of cities sat beside a CSV of marks; today pandas does the lookup properly.

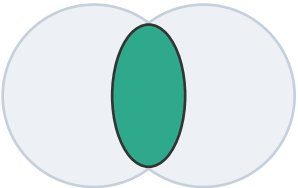
2

The Four Joins

What happens to unmatched rows · 30 minutes

- inner
- left
- right
- outer
- Choosing

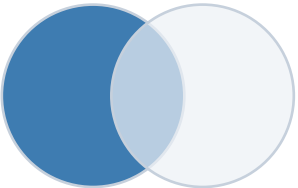
Four joins — the same data, four answers



how="inner"

only matching rows

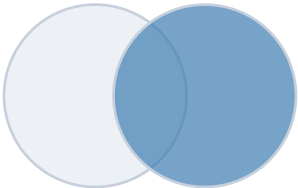
3 rows



how="left"

all of the LEFT table

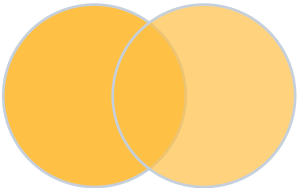
4 rows



how="right"

all of the RIGHT table

4 rows



how="outer"

everything from both

5 rows

how	Keeps	Unmatched become	Use it when
"inner"	rows in BOTH	dropped entirely	you need complete records only
"left"	all left rows	NaN in the right columns	the left table is your population
"right"	all right rows	NaN in the left columns	rare — swap the tables instead
"outer"	everything	NaN on whichever side is missing	auditing: what is in one and not the other

The left join, in detail — the one you will use most

```
students.merge(marks, on="id", how="left")
```

	id	name	city	python
0	1	Ravi	Pune	88.0
1	2	Sara	Mumbai	91.0
2	3	Amit	Delhi	45.0
3	4	Neha	Pune	NaN

Neha kept, with no mark ↑

 Notice the dtype changed

```
before  python  int64
after   python  float64
```

Day 14 again: an integer column cannot hold NaN, so the moment one unmatched row appears, the whole column becomes float. If your IDs or counts suddenly print as 88.0, an unmatched merge is a likely cause.

indicator=True tells you exactly what happened

```
students.merge(marks, on="id",
               how="outer", indicator=True)
# adds a _merge column: both / left_only / right_only
```

Run an outer join with `indicator=True` BEFORE choosing your real join. It answers “how many rows are in one table and not the other?” in a single step — and the answer often changes which join you actually want.

3

Merge in Practice

The arguments you will actually need · 20 minutes

- Different key names
- Overlapping columns
- Merging on the index

Three things that come up every time

Keys with different names

```
students.merge(marks,
               left_on="student_id",
               right_on="id")
# you get BOTH columns - drop one after
```

When the two tables call the same thing by different names — which is most of the time in real data.

⚠ Overlapping column names

```
a.merge(b, on="id")
-> score_x  score_y      unhelpful

a.merge(b, on="id",
        suffixes=("_before", "_after"))
```

If both tables have a score column, pandas keeps both and tags them `_x` and `_y`. Always pass suffixes that mean something.

Merging on the index — and the family of names

```
left.join(right)      on the INDEX, how="left"
a.merge(b, on="id")   on a COLUMN, how="inner"
pd.merge(a, b, on="id") same as a.merge(b, ...)
```

`join` and `merge` do the same job with different defaults: `join` works on the index and keeps all left rows; `merge` works on columns and keeps only matches. Learn `merge` properly and use it everywhere — you can always call `set_index` first if you want index behaviour.

4

Duplicates & Concat

The bug that multiplies your data · 20 minutes

- Row explosion
- validate
- concat

The bug that multiplies your data

Three rows merged with three rows gives... five

orders

cust	amt
1	10
1	20
2	30

customers

cust	name
1	A
1	A-dup
2	B



cust	amt	name
1	10	A
1	10	A-dup
1	20	A
1	20	A-dup
2	30	B

cust 1 appears twice on each side, so $2 \times 2 = 4$ rows come out of it.

Two defences

```
orders.merge(custs, on="cust",
             validate="many_to_one")
```

```
-> MergeError: keys are not
    unique in right dataset
```

```
before = len(orders)
assert len(merged) == before
```

validate states your assumption; the assert proves it held.

Why this is so dangerous

No error, no warning — you simply have more rows than you started with, and every total computed afterwards is inflated. A duplicate in a lookup table is easy to create and invisible once merged. Check `len(df)` before and after every single merge; it costs one line and catches this immediately.

concat — stacking, not matching

Adding ROWS (axis=0, the default)

```
pd.concat([jan, feb, mar], ignore_index=True)

# same columns, more rows - e.g. monthly files
```

⚠ Without `ignore_index` the old labels are kept, so you get index values 0, 0, 0 — and every Day 16 trap returns at once.

```
# a missing column becomes NaN, not an error
```

⚠ axis=1 aligns on the INDEX

```
pd.concat([a, b], axis=1)

# glues columns side by side BY POSITION
```

It matches on the index, not on any key, so two tables whose rows are in different orders will be silently mismatched. If the rows mean something — a customer, a date — use `merge`, not `concat`. `concat` is for tables you already know line up.

The one-line way to decide

Do the two tables describe the SAME things, with different information about each? → `merge`, on a key.

Do they describe DIFFERENT things, with the same information about each? → `concat`, stacking rows.

Twelve monthly sales files are a `concat`. Sales plus a customer lookup is a `merge`.

Debug corner — eight merge bugs

You wrote	What happens	Fix
<code>a.merge(b, on="id")</code>	rows silently dropped — inner is the default	<code>how="left"</code>
<code>duplicate keys in the lookup</code>	MORE rows out than in	<code>validate="many_to_one"</code>
<code>no row-count check</code>	inflated totals reach the report	<code>assert len(after) == len(before)</code>
<code>int column now shows 88.0</code>	an unmatched row forced NaN in	expected after a left join
<code>score_x and score_y</code>	both tables had that column	<code>suffixes=("_a", "_b")</code>
<code>concat without ignore_index</code>	duplicate index labels 0, 0, 0	<code>ignore_index=True</code>
<code>concat(axis=1) on unsorted rows</code>	silently mismatched — aligns on index	<code>merge(on=key)</code>
<code>merged["id"] missing</code>	keys had different names	<code>left_on=... right_on=...</code>

Rows 1 to 3 are all the same habit: check how many rows you have, before and after. All eight are in today's notebook.

Putting it together — three tables into one report

```
import pandas as pd

students = pd.read_csv("students.csv")  # id, name, city
marks    = pd.read_csv("marks.csv")    # id, python, stats
cities   = pd.read_csv("cities.csv")    # city, state

# ---- 0. look BEFORE you join
check = students.merge(marks, on="id", how="outer", indicator=True)
print(check["_merge"].value_counts())

# ---- 1. keep every student, even those with no marks
before = len(students)
df = students.merge(marks, on="id", how="left",
                    validate="one_to_one")
assert len(df) == before, "the merge changed the row count!"

# ---- 2. add the state from a lookup table
df = df.merge(cities, on="city", how="left", validate="many_to_one")
assert len(df) == before

# ---- 3. report what is missing, do not hide it
print("students with no marks:", df["python"].isna().sum())
print("cities with no state  :", df["state"].isna().sum())

# ---- 4. now it is safe to summarise
print(df.groupby("state")["python"].mean().round(1))
```

Two asserts and one indicator. That is the difference between a report you can defend and one you merely hope is right.



Outer + indicator first

see the damage before choosing



how="left"

every student survives



validate=

states your assumption out loud



assert on len

catches the row explosion



one_to_one vs many_to_one

students unique; cities repeat



Report the NaNs

Day 14's rule, still applying



Then groupby

yesterday's tool, on joined data

Practice session — 30 minutes



Predict the output

Row counts, NaNs, dtypes, suffixes

8 questions



Spot & fix the bug

Silent drops, explosion, concat misuse

4 questions



Write the code

Join, choose how, audit, stack, lookup

5 tasks

Your notebook

Day18_Practice_
Questions.ipynb

- print len() before and after
- Ask: what happens to non-matches?
- Raise a hand — I am walking around
- Solutions shared after class

Every merge in this notebook must be followed by a row-count check. If the number changed, say why.

TODAY IN TWELVE LINES

Recap

- 1 merge combines two tables on a shared key.
- 2 The real question is what happens to rows that DON'T match.
- 3 inner keeps matches only — and it is the DEFAULT.
- 4 left keeps every left row, filling the rest with NaN.
- 5 outer keeps everything; right is rare — swap the tables.
- 6 An unmatched row turns an int column into float.
- 7 indicator=True adds both / left_only / right_only.
- 8 Run outer + indicator FIRST, then pick your real join.
- 9 ⚠ Duplicate keys multiply rows. 3 and 3 can give 5.
- 10 validate="many_to_one" states your assumption.
- 11 assert len(after) == len(before) after every merge.
- 12 concat stacks; use ignore_index=True. axis=1 aligns on index.

Homework

Take two CSVs and run all four joins, recording the row count of each · Deliberately duplicate a key and watch the row count explode · Rebuild Day 9's city lookup as a merge instead of a dict

Next class — Topic 1.18

Data cleaning: missing values & duplicates — finding them, deciding what to do, and the cost of each choice